

Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

Sta. Mesa, Manila

LAlgo

Language Design Specification

A Domain-Specific Language for Observable Execution,
Tracing, and Reasoning in Algorithms

Formal Syntax and Operational Lexicon

Submitted by: Group 4

Buenaventura, Harvey L.

De Mesa, Joshua C.

Dela Rosa, Mico Von Adriel A.

Magalona, Adriel M.

Morales, Timothy Gabriel

Nevado, Sean Andrew M.

Salvacion, Dean James G.

Submitted to

Prof. Ria A. Sagum

Course and Subject Code

Principles of Programming Languages (COSC 303)

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Inspiration . . . 1
- 1.3 Problem Addressed . . . 1
- 1.4 Objectives . . . 2

2 Lexical Elements **2**
- 2.1 Character Set . . . 2
- 2.2 Identifiers . . . 2
- 2.3 Finite Automata (Lexical Analyzer Execution Model) . . . 3
  - 2.3.1 Master Lexer Dispatch Automaton . . . 4
  - 2.3.2 DFA for Identifiers and Keywords . . . 5
  - 2.3.3 DFA for Numeric Literals (Integer and Decimal) . . . 5
  - 2.3.4 DFA for Relational and Assignment Operators . . . 5
  - 2.3.5 DFA for Comments and Division . . . 6
  - 2.3.6 DFA for String Literals . . . 6
  - 2.3.7 DFA for Character Literals . . . 7
  - 2.3.8 DFA for Whitespace (Blanks) . . . 7
  - 2.3.9 DFA for Delimiters and Brackets . . . 7
  - 2.3.10 Representative DFA for Keyword Recognition . . . 8
  - 2.3.11 Scope and Limitations . . . 8
- 2.4 Operation Symbols . . . 9
  - 2.4.1 Arithmetic Operators . . . 9
  - 2.4.2 Boolean (Logical) Operators . . . 9
  - 2.4.3 Relational Comparison Operators . . . 9
- 2.5 Keywords and Reserved Words . . . 10
- 2.6 Noise Words . . . 10
- 2.7 Comments . . . 10
  - 2.7.1 Single-Line Comments . . . 11
  - 2.7.2 Multi-Line Comments . . . 11
- 2.8 Blanks (Whitespace Handling) . . . 11
- 2.9 Delimiters and Brackets . . . 11
- 2.10 Free-Field Format . . . 12
- 2.11 Expressions . . . 12
  - 2.11.1 Operator Precedence and Associativity . . . 12
- 2.12 Primitive Data Types . . . 12
  - 2.12.1 Integer . . . 13
  - 2.12.2 Decimal . . . 13
  - 2.12.3 Character . . . 13
  - 2.12.4 Boolean . . . 13
  - 2.12.5 Abstract Data Types (Optional / Supported) . . . 13
- 2.13 Statements . . . 14
  - 2.13.1 Declarative Statement (Optional) . . . 14
  - 2.13.2 Input Statement . . . 14
  - 2.13.3 Output Statement . . . 14
  - 2.13.4 Assignment Statement . . . 14
  - 2.13.5 Conditional Statement . . . 15
  - 2.13.6 Loop (Iterative) Statements . . . 15
  - 2.13.7 Future Reserved Extensions . . . 15

1 Introduction

1.1 Overview

LAlgo is a proposed imperative domain-specific language (DSL) for expressing data structures and algorithms (DSA) in readable pseudocode. This proposal focuses on the lexical analyzer: the character alphabet, token classes, and finite automata used to recognize source text. A complete parser and runtime are future work.

The examples use familiar declarations, operators, indexing, and control-flow words to show the intended token vocabulary. They illustrate how an algorithm such as binary search could be written, but they do not claim that execution tracing or invariant checking is implemented by the present lexical proposal.

1.2 Inspiration

The primary inspiration for LAlgo emerges from the cognitive disconnect students encounter when transitioning between algorithmic pseudocode and practical code implementation. Textbooks routinely present algorithms alongside step-by-step state trace tables and formal loop assertions. However, translating these designs into standard programming environments forces students to manage language-specific boilerplate, unmonitored variable states, and implicit type conversions that mask algorithmic errors.

When a learner encounters an off-by-one error, the sequence of intermediate states can be difficult to reconstruct. LAlgo draws inspiration from educational pseudocode, Pascal-like syntax, and structured tracing tools. This paper establishes the lexical groundwork for that longer-term direction without specifying a tracing runtime.

1.3 Problem Addressed

Traditional programming language compilers and runtime engines are intentionally opaque: they consume source code, optimize away intermediate variables, and produce final output. Under this execution model:



This pipeline introduces several core educational problems:

- **Silent Intermediate States.** Students observe only terminal outputs, leaving internal invariant breakdowns, element swaps, and path traversals invisible.
- **Unverified Invariants.** Algorithmic correctness proofs rely on inductive loop invariants, yet standard languages provide no native, readable syntactic construct to express whether loop conditions remain valid across iterations.
- **Hidden State Changes.** Intermediate variable updates and branch decisions can be difficult to follow without manually adding diagnostic output.

The present proposal addresses the lexical foundation of this problem. Parsing, execution-state observation, and any later tracing or verification behavior require separate specifications.

1.4 Objectives

The primary goal of this iteration is to define an unambiguous token vocabulary and deterministic lexical recognition model for algorithmic source text. Specifically, the project aims to:

1. Define an unambiguous lexical structure and formal automata suited for algorithmic problem solving.
2. Recognize tokens used by illustrative declarations, I/O, assignments, conditional branches, and loops without defining their full parser or runtime behavior.
3. Recognize collection-type words (`ARRAY` and `STRING`) and the associated delimiters.
4. Reserve `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` for possible future syntactic extensions, without assigning them executable behavior in this iteration.

2 Lexical Elements

2.1 Character Set

LAlgo utilizes a standard ASCII-based character set denoted by the finite alphabet Σ. The alphabet is partitioned into distinct lexical classes:

$$\Sigma = \Sigma_{\text{alpha}} \cup \Sigma_{\text{digits}} \cup \Sigma_{\text{arithmetic}} \cup \Sigma_{\text{relational}} \cup \Sigma_{\text{delimiters}} \cup \Sigma_{\text{underscore}} \cup \Sigma_{\text{quotes}} \cup \Sigma_{\text{whitespace}} \cup \Sigma_{\text{literal}}.$$

The supported character categories are explicitly defined below.

Alphabetic Characters (Σ_{alpha}) `a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`

Numeric Digits (Σ_{digits}) `0 1 2 3 4 5 6 7 8 9`

Arithmetic Symbols ($\Sigma_{\text{arithmetic}}$) `+ - * / % ^`

Relational and Assignment Symbols ($\Sigma_{\text{relational}}$) `= ! < >`

Delimiters and Punctuation ($\Sigma_{\text{delimiters}}$) `( ) [ ] { } , ; : .`

Identifier Continuation Symbol ($\Sigma_{\text{underscore}}$) `_`

Quotation Symbols (Σ_{quotes}) `" '`

Whitespace Characters ($\Sigma_{\text{whitespace}}$) Space (ASCII 32), horizontal tab (ASCII 9), and line feed (ASCII 10).

Literal-Only Symbols (Σ_{literal}) Any remaining printable ASCII character (ASCII 32–126) other than backslash; these symbols are permitted only inside quoted literals.

The scanner combines the individual relational characters into `==`, `!=`, `<=`, and `>=` using lookahead. These are multi-character operators rather than additional members of the character alphabet; `!` alone is not a valid operator.

2.2 Identifiers

Identifiers are programmer-defined tokens used to name variables, constants, and algorithmic routines. An identifier must satisfy the following lexical constraints:

- It must begin with an alphabetic character (`a-z` or `A-Z`).
- It may subsequently contain alphabetic characters, numeric digits (`0-9`), or underscores (`_`).
- It must not match any reserved keyword or noise word.
- It is strictly case-sensitive: `counter`, `Counter`, and `COUNTER` are distinct tokens.

Formally, the lexical structure of an identifier is expressed by the regular expression:

```
[a-zA-Z][a-zA-Z0-9_]*
```

Valid identifiers

```
low
high
binarySearch
target_index
arraySize2
```

Invalid identifiers

```
2low         // Begins with a digit
target-index // Hyphen means subtraction
WHILE        // Reserved keyword
SET          // Reserved noise word
```

2.3 Finite Automata (Lexical Analyzer Execution Model)

LAlgo's lexical scanner processes raw source code using a set of deterministic finite automata (DFAs). Formally, each automaton is defined as a five-tuple:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where Q is the finite set of internal states, Σ is the input character alphabet, $\delta : Q \times \Sigma \to Q$ is the state transition function, $q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.

2.3.1 Master Lexer Dispatch Automaton

The lexical analyzer reads the first character of each lexeme at q_0 and dispatches to the matching token recognizer. The identifier, numeric, relational, string, comment/division, delimiter, arithmetic, and character paths are shown below. Each recognizer applies the **maximal munch (longest match)** principle where needed, emits a token after reaching an accepting state, and returns control to q_0. Whitespace loops at q_0 and is discarded; scanning stops at End-of-File (EOF).

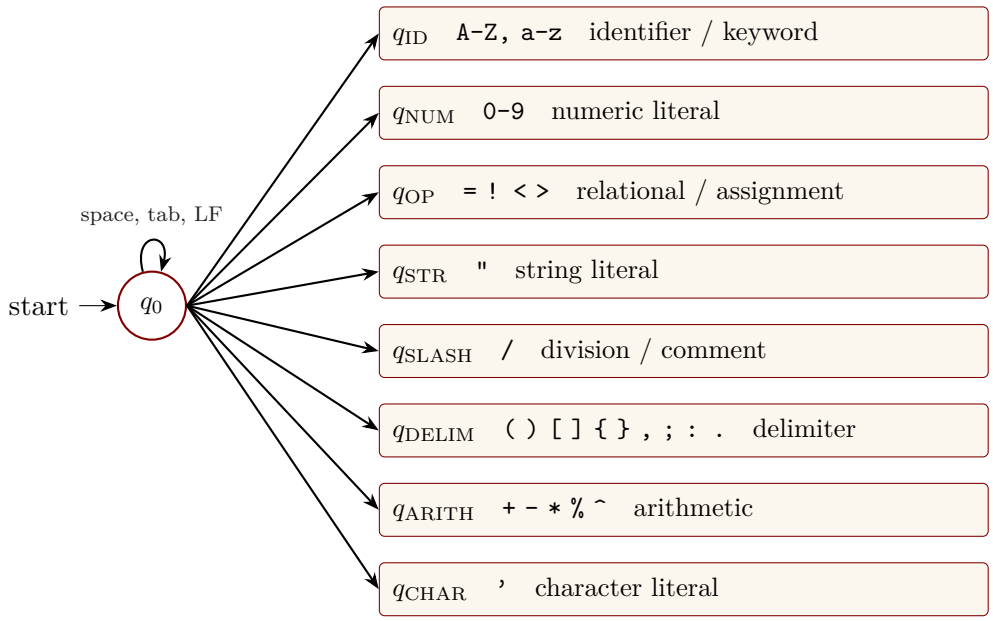


Figure 1: Master lexer dispatch routes from q_0 to the appropriate token recognizer.

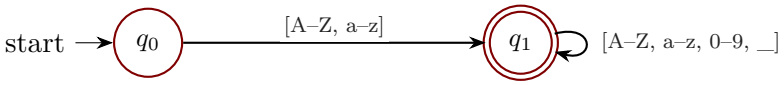
Step-by-step lexical scan trace. Consider the statement `count = 10`.

Step	Current state	Character(s)	Automaton action	Token emitted
1	q_0	`'c'`	Transition to q_{ID}	None
2	q_{ID}	`'o', 'u', 'n', 't'`	Self-loop on q_{ID}	None
3	q_{ID}	space	No transition; emit token and reset	`TOKEN(IDENT, "count")`
4	q_0	space	Discard whitespace self-loop	None
5	q_0	`'='`	Transition to q_{OP}	None
6	q_{OP}	space	Lookahead is not `'='`; emit and reset	`TOKEN(ASSIGN, "=")`
7	q_0	space	Discard whitespace self-loop	None
8	q_0	`'1'`	Transition to q_{NUM}	None
9	q_{NUM}	`'0'`	Self-loop on q_{NUM}	None
10	q_{NUM}	EOF	Buffer end; emit and reset	`TOKEN(INTEGER, 10)`

2.3.2 DFA for Identifiers and Keywords

Identifiers begin with an alphabetic character and may continue with letters, digits, or underscores. Keywords have the same lexical morphology and are subsequently resolved using an internal reserved-word symbol table.

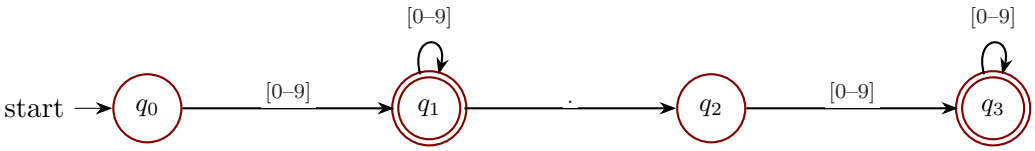
- $Q = \{q_0, q_1\}$
- q_0: initial state
- $F = \{q_1\}$ (accepting state)



2.3.3 DFA for Numeric Literals (Integer and Decimal)

This automaton distinguishes whole-number values (`INTEGER`) from fractional real numbers (`DECIMAL`) through decimal-point transitions.

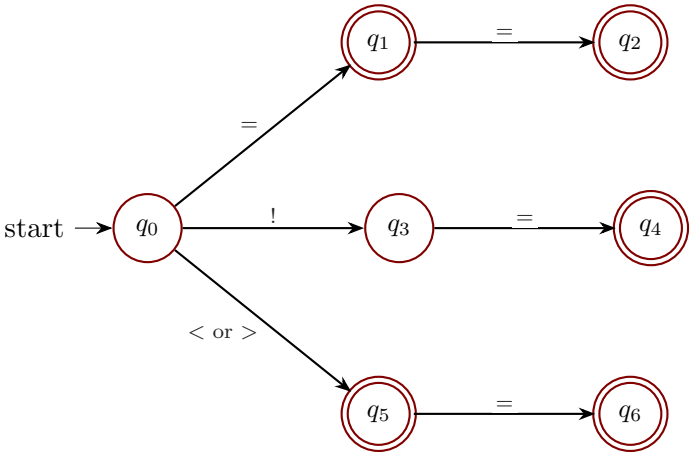
- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_1, q_3\}$, where q_1 accepts `INTEGER` and q_3 accepts `DECIMAL`



2.3.4 DFA for Relational and Assignment Operators

This DFA disambiguates single-character assignment and comparison operators (`=`, `<`, `>`) from compound relational operators (`==`, `!=`, `<=`, `>=`).

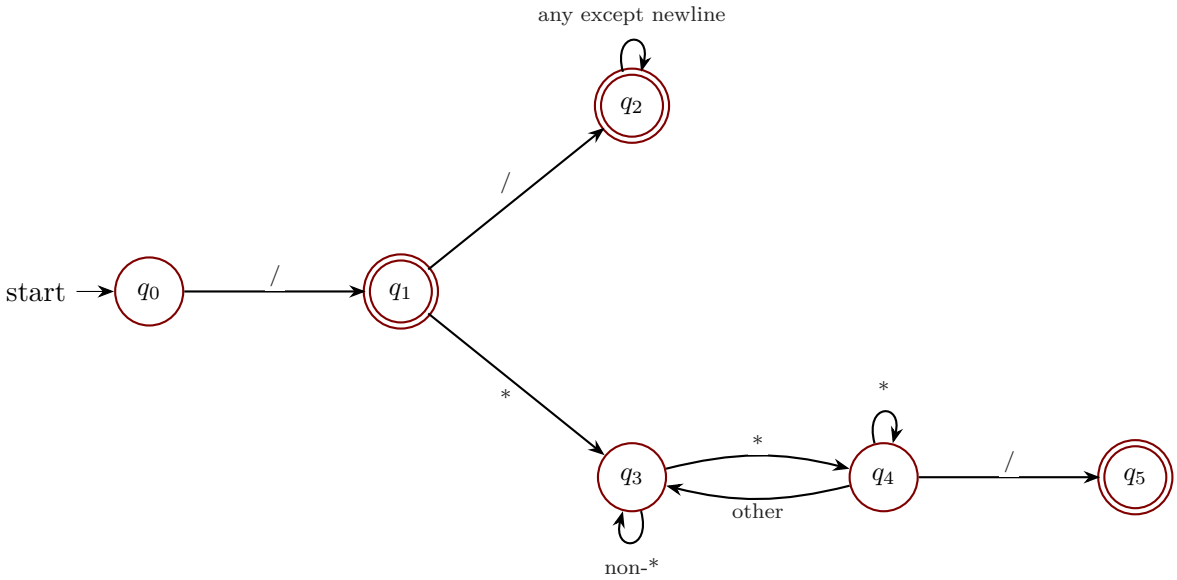
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$
- $F = \{q_1, q_2, q_4, q_5, q_6\}$



2.3.5 DFA for Comments and Division

This DFA distinguishes division (`/`) from single-line comments (`//`) and block comments (`/* ... */`) using lookahead scanning.

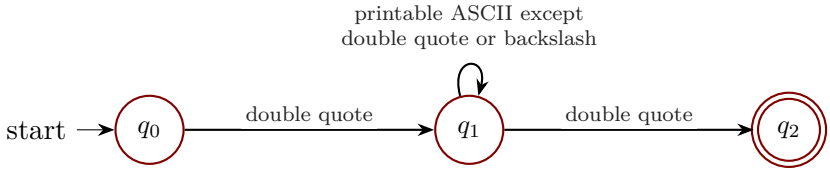
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
- $F = \{q_1, q_2, q_5\}$, where q_1 accepts division, q_2 accepts a single-line comment, and q_5 accepts a multi-line comment



2.3.6 DFA for String Literals

This DFA recognizes character-string literals enclosed by double quotation marks (`"..."`). Their contents consist of printable ASCII characters other than a double quote or backslash; escape sequences and embedded newlines are not defined.

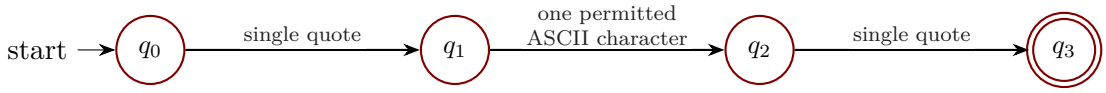
- $Q = \{q_0, q_1, q_2\}$
- $F = \{q_2\}$



2.3.7 DFA for Character Literals

A character literal consists of one printable ASCII character enclosed in single quotation marks, such as `'A'` or `','`. The character between the quotes may be ASCII 32–126 except a single quote or backslash; escape sequences are not defined in this specification. An empty, multi-character, or unterminated literal is a lexical error.

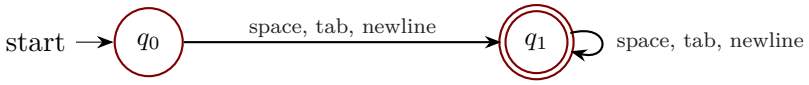
- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_3\}$



2.3.8 DFA for Whitespace (Blanks)

Whitespace characters—spaces, horizontal tabs, and line feeds—are recognized by a trivial single-loop automaton. Tokens matched by this automaton are consumed by the lexer but are not emitted to the parser; they exist only to separate adjacent lexemes.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$ (accepting, but discarded)

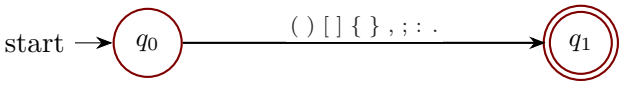


Because whitespace carries no semantic weight (Section 2.8), the scanner discards any accepted whitespace lexeme instead of forwarding it to the parser, then resumes scanning from q_0.

2.3.9 DFA for Delimiters and Brackets

Delimiters and brackets are single-character tokens: `( ) [ ] { } , ; : .`. Because no lookahead is required, each is accepted after exactly one transition from q_0.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$, where the specific delimiter determines the emitted token type (for example, `LPAREN`, `RBRACE`, or `COMMA`)



Unlike the multi-character automata above, no maximal munch is needed: the lexer emits the corresponding token immediately after the single accepted transition and resets to q_0.

2.3.10 Representative DFA for Keyword Recognition

The identifier DFA defines the shape shared by identifiers and reserved words: a letter followed by zero or more letters, digits, or underscores. The automaton below illustrates how that general shape narrows into the exact spelling of **IF**, **INPUT**, **INTEGER**, and **INVARIANT**. The first three are current language keywords; **INVARIANT** is reserved for a future extension. They share the prefix `I` and diverge as their spellings differ.

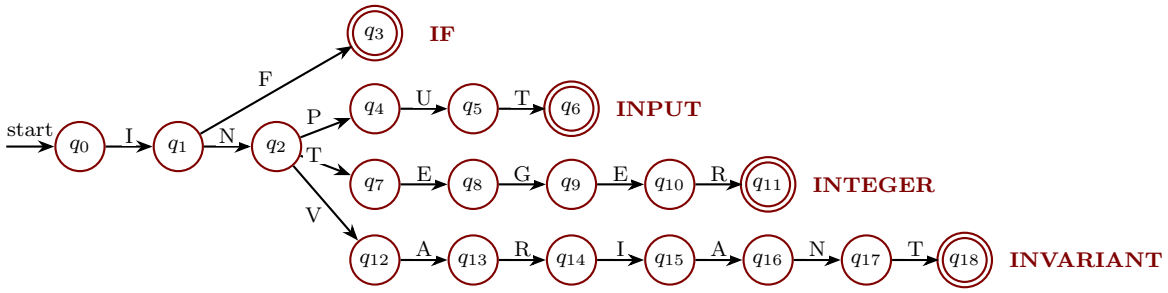


Figure 2: Shared-prefix keyword recognition with non-overlapping branches and explicit accepting states.

At each accepting state (q_3, q_6, q_{11}, and q_{18}), if the following character continues the identifier pattern `[a-zA-Z0-9_]` instead of a delimiter or whitespace, the scanner does not emit a keyword token. It falls back into the general identifier state q_{ID}. For example, `IFrame` does not match **IF** because `r` extends the identifier. Any unshown letter at a branch likewise routes to q_{ID}.

> **Note.** The same shared-prefix principle extends to other current keywords, future-reserved words, and noise words. For example, **ALGORITHM**, **ARRAY**, and **AND** share an A prefix, while **TRACE**, **TRUE**, and **THE** share a T prefix. The scanner uses table lookup after the general identifier DFA accepts a lexeme; this diagram is only representative.

2.3.11 Scope and Limitations

This finite-automata model operates strictly at the lexical level, where tokens are recognized and classified. It does not check type rules, declaration-before-use, statement structure, or grammar-level correctness. Those checks belong to parsing and semantic analysis.

Several simplifications keep the model clear and within the scope of lexical analysis:

- **Error handling.** The error state is implicit. An unterminated string literal, character literal, or block comment ends in a non-accepting state and is reported as a lexical error.
- **Keyword resolution.** The keyword diagram is pedagogical. The actual scanner resolves keywords through table lookup after the identifier DFA accepts a lexeme, rather than maintaining a physically separate branch for every reserved word.
- **Future-reserved words.** **ALGORITHM**, **RETURN**, **TRACE**, and **INVARIANT** are recognized as reserved lexemes but have no grammar or runtime semantics in this iteration.
- **Nested brackets.** At the lexical level, `{`, `}`, `(`, `)`, `[`, and `]` are individual delimiter tokens. Nesting and matching are parser responsibilities.
- **Newlines.** CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted.
- **Block comments.** Block comments do not nest; the first `*/` closes the comment.
- **Signed numbers.** Negative numbers are formed by a subtraction operator followed by a literal, so the numeric literal itself contains no sign.

- **Number-identifier boundaries.** A numeric literal immediately followed by a letter (for example, `5thSample`) becomes an `INTEGER` literal followed by an identifier; the parser rejects the resulting pair.

2.4 Operation Symbols

LAlgo defines seven arithmetic operators, three core logical operators, and six relational comparison operators. The descriptions below indicate intended meanings; they do not constitute a runtime implementation.

2.4.1 Arithmetic Operators

LAlgo provides seven foundational arithmetic operations:

Operation	Operator	Associativity	Semantic description
Addition	`+`	Left	Computes the sum of operands.
Subtraction	`-`	Left	Computes the difference of operands.
Multiplication	`*`	Left	Computes the product of operands.
Division	`/`	Left	Computes a real quotient (`DECIMAL`).
Modulo	`%`	Left	Remainder after integer division.
Integer division	`DIV`	Left	Returns the truncated integer quotient.
Exponentiation	`^`	Right	Raises the base to a power.

Distinction between `/` and `DIV`:

```
INTEGER a = 7
INTEGER b = 2
OUTPUT a / b   // Yields Decimal: 3.5
OUTPUT a DIV b // Yields Integer: 3
OUTPUT a % b   // Yields Integer remainder: 1
```

Exponentiation example:

```
OUTPUT 2 ^ 3 // Yields 8
```

2.4.2 Boolean (Logical) Operators

Boolean expressions use three logical keywords. This lexical specification identifies their tokens; evaluation strategy, including any short-circuit behavior, is outside its scope.

Operation	Operator	Precedence	Semantic description
Logical NOT	`NOT`	Highest	Unary negation; inverts the truth value.
Logical AND	`AND`	Medium	Conjunction; `TRUE` only if both operands are `TRUE`.
Logical OR	`OR`	Lowest	Disjunction; `TRUE` if at least one operand is `TRUE`.

2.4.3 Relational Comparison Operators

Comparison operators evaluate the quantitative relationship between expressions and yield a strictly typed `BOOLEAN` value (`TRUE` or `FALSE`).

Comparison	Symbol	Example
Equal to	==	`mid == target`
Not equal to	!=	`count != 0`
Less than	<	`low < high`
Greater than	>	`score > 75`
Less than or equal to	<=	`i <= length`
Greater than or equal to	>=	`val >= 0`

2.5 Keywords and Reserved Words

Reserved words are recognized by the lexer and cannot be used as identifiers. The current language keywords and the words held for future extensions are distinguished below. All use uppercase canonical spelling.

Category	Reserved words
Primitive types	INTEGER, DECIMAL, CHARACTER, BOOLEAN
Collection types	ARRAY, STRING
Control flow	IF, ELSE, WHILE, FOR
I/O statements	INPUT, OUTPUT
Boolean literals	TRUE, FALSE
Arithmetic operator	DIV
Logical operators	AND, OR, NOT
Future extensions (reserved only)	ALGORITHM, RETURN, TRACE, INVARIANT

The table contains 18 current language keywords and four future-reserved words. The latter are recognized as distinct tokens but are not valid executable constructs in this proposal. The separate noise words SET and THE are also reserved.

2.6 Noise Words

Noise words are optional lexical tokens designed to mirror natural-language pseudocode. They enhance readability for beginner programmers but carry no operational weight. The parser permits them only in the positions shown below and omits them from the Abstract Syntax Tree (AST). They remain reserved to prevent identifier ambiguity.

- SET: optional prefix before an assignment.

```
SET index = 0 // Semantically identical to: index = 0
```

- THE: optional readability marker in output routines.

```
OUTPUT THE result // Semantically identical to: OUTPUT result
```

2.7 Comments

Comments allow programmers to document algorithmic logic, state specifications, and preconditions. The lexical scanner strips comments before parsing, ensuring zero execution overhead.

2.7.1 Single-Line Comments

Single-line comments begin with two forward slashes (`//`) and continue until the next newline.

Regular expression:

```
//[^\n]*
```

Example:

```
// Initialize the lower search boundary
INTEGER low = 0
```

2.7.2 Multi-Line Comments

Multi-line comments are delimited by `/*` and `*/`, allowing block documentation across multiple lines.

Regular expression:

```
/\*[\s\S]*?\*/
```

Example:

```
/*
Binary Search Implementation
Precondition: Input array must be sorted in ascending order.
Returns: Index of target element, or -1 if not found.
*/
```

2.8 Blanks (Whitespace Handling)

Whitespace includes spaces (ASCII 32), horizontal tabs (ASCII 9), and line feeds (ASCII 10). CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted. LAlgo treats whitespace strictly as a non-semantic token separator:

- Whitespace demarcates adjacent keywords, identifiers, and literals.
- Consecutive whitespace characters collapse into one lexical boundary.
- Indentation carries no structural or scoping significance, preventing syntax errors caused by mixed tabs and spaces.

These lines are lexically equivalent:

```
INTEGER x = 100;
```

```
INTEGER       x    =    100;
```

2.9 Delimiters and Brackets

Delimiters define statement boundaries, parameter passing, collection bounds, and hierarchical scoping blocks.

- **Parentheses `( )`:** enclose mathematical expressions, conditional guards, and algorithm parameters.

- **Square brackets `[ ]`:** delimit array literals and specify indexed element access.
- **Curly braces `{ }`:** delimit compound statement blocks, loops, conditional branches, and algorithm bodies.
- **Comma `,`:** separates arguments, parameters, and array elements.
- **Semicolon `;`:** is recognized as a separate token. Rules for using it as a statement separator belong to the future parser specification.

2.10 Free-Field Format

LAlgo uses a free-field format. Program source code is not bound to fixed columns, physical margins, or indentation rules.

- **Block markers.** Opening and closing braces are separate tokens in the examples; their nesting and scope effects belong to a future parser specification.
- **Statement formatting.** Outside comments and quoted literals, spaces, tabs, and line feeds separate tokens without defining statement boundaries. Whether tokens form one or multiple statements is a parser-level question.

Illustrative free-field source:

```
INTEGER low = 0; INTEGER high = 10;
IF (low < high) { OUTPUT low; }
```

2.11 Expressions

The examples below show token sequences for intended arithmetic, relational, and logical expressions. Their complete grammar, typing, and evaluation rules are outside this lexical proposal.

1. **Arithmetic expressions** evaluate to numeric types (`INTEGER` or `DECIMAL`).

   ```
   (low + high) DIV 2
   ```

2. **Relational expressions** compare values to produce a truth condition.

   ```
   numbers[mid] > target
   ```

3. **Logical expressions** combine Boolean expressions using propositional connectives.

   ```
   (low <= high) AND (found == FALSE)
   ```

2.11.1 Operator Precedence and Associativity

Evaluation follows the standard hierarchy below, ordered from highest to lowest precedence.

2.12 Primitive Data Types

LAlgo supports four fundamental primitive types and two built-in abstract data types suited for algorithmic collections.

Rank	Operator	Description	Associativity
1	`( ) [ ]`	Grouping and array indexing	Left-to-right
2	`^`	Exponentiation	Right-to-left
3	`NOT`, `-` (unary)	Logical negation and sign inversion	Right-to-left
4	`* / %`, `DIV`	Multiplicative operations	Left-to-right
5	`+ -`	Additive operations	Left-to-right
6	`< <= > >=`	Relational inequalities	Left-to-right
7	`== !=`	Relational equalities	Left-to-right
8	`AND`	Logical conjunction	Left-to-right
9	`OR`	Logical disjunction	Left-to-right

2.12.1 Integer

Represents whole numbers: positive, negative, or zero.

```
INTEGER count = 0
INTEGER step = -5
```

2.12.2 Decimal

Represents numbers containing fractional values.

```
DECIMAL ratio = 0.618
DECIMAL total = -12.50
```

2.12.3 Character

Represents one printable ASCII character literal enclosed in single quotation marks, following the character-literal DFA in Section 2.3.7.

```
CHARACTER grade = 'A'
CHARACTER delimiter = ','
```

2.12.4 Boolean

Represents binary truth values defined by the literals `TRUE` and `FALSE`.

```
BOOLEAN isFound = FALSE
BOOLEAN hasTerminated = TRUE
```

2.12.5 Abstract Data Types (Optional / Supported)

To support data-structure modeling, LAlgo provides two native collection types:

- **String.** An immutable ordered sequence of characters enclosed in double quotation marks.

  ```
  STRING algorithmName = "Binary Search"
  ```

- **Array.** A contiguous, zero-indexed collection of homogeneous elements.

  ```
  ARRAY numbers = [2, 4, 6, 8, 10]
  ```

```
INTEGER firstElement = numbers[0]
```

2.13 Statements

The following snippets illustrate token sequences for proposed declarations and procedural statements. They do not define a complete statement grammar or runtime semantics. The four future-reserved words are not used as executable statements here.

2.13.1 Declarative Statement (Optional)

Declarative statements use a type keyword followed by an identifier, optionally with an initializing expression. Default values for uninitialized variables are not specified by this lexical proposal.

```
INTEGER minIndex
DECIMAL threshold = 0.05
```

2.13.2 Input Statement

The INPUT statement is followed by a target identifier. Input conversion and runtime behavior are outside the scope of this lexical proposal.

```
INTEGER target
INPUT target
```

2.13.3 Output Statement

The illustrated OUTPUT form is followed by an expression. Output formatting and runtime delivery are not defined by the lexer.

```
OUTPUT "Search completed at index:"
OUTPUT targetIndex
```

2.13.4 Assignment Statement

The assignment symbol = is recognized between a target identifier or indexed value and an expression. Assignment evaluation is outside the lexical model.

```
low = mid + 1
numbers[0] = 99
```

2.13.5 Conditional Statement

Conditional statements provide two-way and multi-way branch selection through `IF`, optional `ELSE IF`, and `ELSE` blocks.

```
IF (numbers[mid] == target) {
    OUTPUT "Target Found"
    found = TRUE
}
ELSE IF (numbers[mid] < target) {
    low = mid + 1
}
ELSE {
    high = mid - 1
}
```

2.13.6 Loop (Iterative) Statements

The examples use `WHILE` and `FOR` tokens to illustrate indeterminate and determinate iteration forms.

WHILE loop (illustrative binary search). The `WHILE` form below shows a boundary update in both comparison directions. The `INVARIANT` token is reserved for future work and is intentionally absent from this example.

```
ARRAY numbers = [2, 4, 6, 8, 10]
INTEGER target = 8
INTEGER low = 0
INTEGER high = 4
INTEGER mid = 0
BOOLEAN found = FALSE
WHILE (low <= high AND found == FALSE) {
    mid = low + (high - low) DIV 2
    // Update search boundaries according to the comparison
    IF (numbers[mid] == target) {
        found = TRUE
        OUTPUT mid
    }
    ELSE IF (numbers[mid] < target) {
        low = mid + 1
    }
    ELSE {
        high = mid - 1
    }
}
```

FOR loop. The `FOR` construct packages counter initialization, loop-condition checking, and step updates into a single header.

```
FOR (INTEGER i = 0; i < 10; i = i + 1) {
    OUTPUT numbers[i]
}
```

2.13.7 Future Reserved Extensions

`ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` are reserved lexemes only. This proposal does not define their parser productions, value rules, tracing schedule, invariant-failure behavior, or other runtime effects. They are intentionally omitted from the illustrative snippets above.